

## **4 BANCO DE DADOS**

O banco de dados do sistema TrinixTech foi estruturado para atender às funcionalidades de uma aplicação web voltada à divulgação e à gestão de serviços de assistência técnica. A base de dados organiza informações relacionadas a clientes, usuários do sistema, produtos em estoque, vendas, ordens de serviço e itens utilizados em cada atendimento. Dessa forma, o banco atua como elemento central para garantir o armazenamento, a recuperação e a consistência das informações manipuladas pela aplicação.

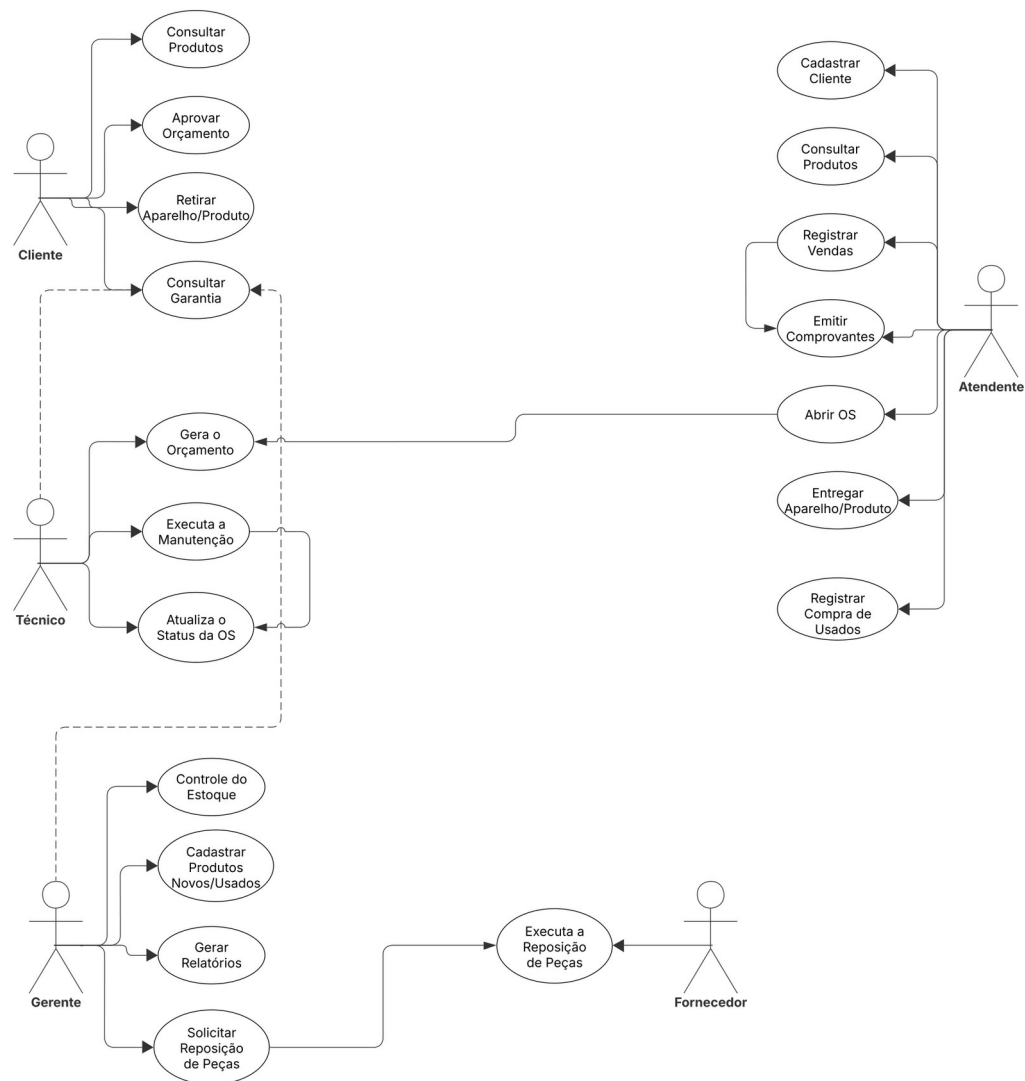
A modelagem foi elaborada considerando os processos principais identificados no sistema, como cadastro de clientes, consulta de produtos, registro de vendas, abertura de ordens de serviço, atualização do status da manutenção, controle de estoque, geração de relatórios e solicitação de reposição de peças. Esses processos são representados no diagrama de caso de uso e fundamentam a definição das entidades persistidas no banco de dados.

### **4.1 Relação entre casos de uso e persistência de dados**

O diagrama de caso de uso demonstra que diferentes atores interagem com o sistema, entre eles Cliente, Atendente, Técnico, Gerente e Fornecedor. O Atendente realiza operações como cadastrar cliente, registrar vendas, emitir comprovantes e abrir ordens de serviço. O Técnico gera orçamento, executa a manutenção e atualiza o status da ordem de serviço. O Gerente realiza controle de estoque, cadastro de produtos, geração de relatórios e solicitação de reposição de peças.

A partir desses requisitos, foram definidas tabelas específicas para clientes, produtos, vendas, itens de venda, ordens de serviço, itens de ordem de serviço, perfis de usuário e papéis de acesso. Assim, cada funcionalidade do sistema passa a ter suporte direto na estrutura do banco de dados, reduzindo inconsistências e facilitando consultas futuras.

Figura 1 - Diagrama de caso de uso do sistema TrinixTech



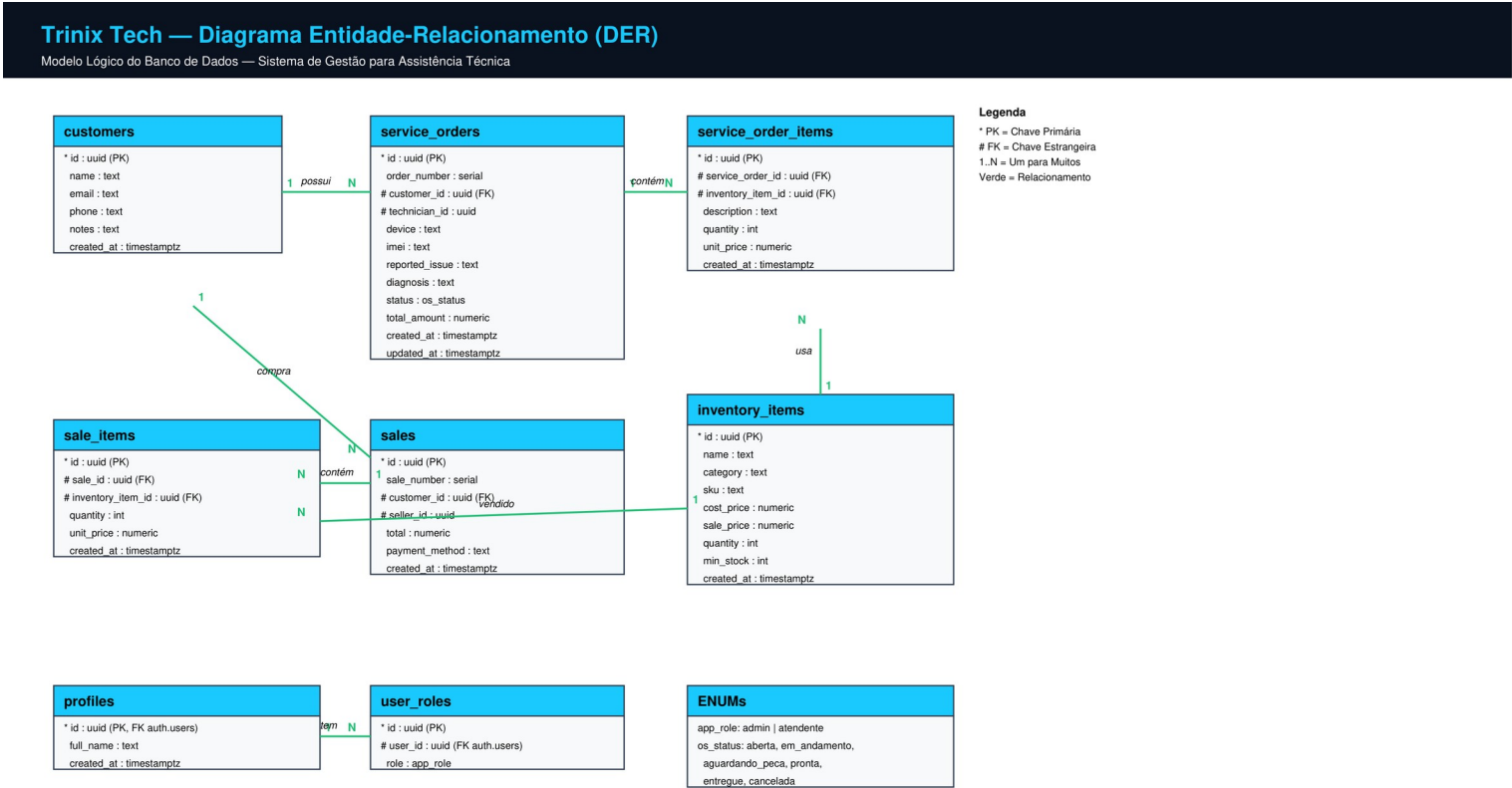
Fonte: Elaborado pelos autores (2026).

## 4.2 Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento (DER) apresenta a visão conceitual da estrutura de dados do TrinixTech. Ele evidencia as principais entidades do sistema, seus atributos, as chaves primárias, as chaves estrangeiras e as cardinalidades existentes entre as tabelas. A modelagem considera relacionamentos do tipo um para muitos, como um cliente associado a várias vendas ou ordens de serviço, e uma venda composta por vários itens.

No DER, a entidade `customers` representa os clientes atendidos pela assistência técnica. A entidade `service_orders` representa as ordens de serviço abertas para manutenção de aparelhos. A entidade `inventory_items` armazena os produtos e peças disponíveis em estoque. Já `sales` e `sale_items` registram vendas e seus respectivos itens. Também são previstas entidades relacionadas ao controle de acesso, como `profiles` e `user_roles`, integradas ao mecanismo de autenticação da aplicação.

Figura 2 - Diagrama Entidade-Relacionamento do banco de dados TrinixTech

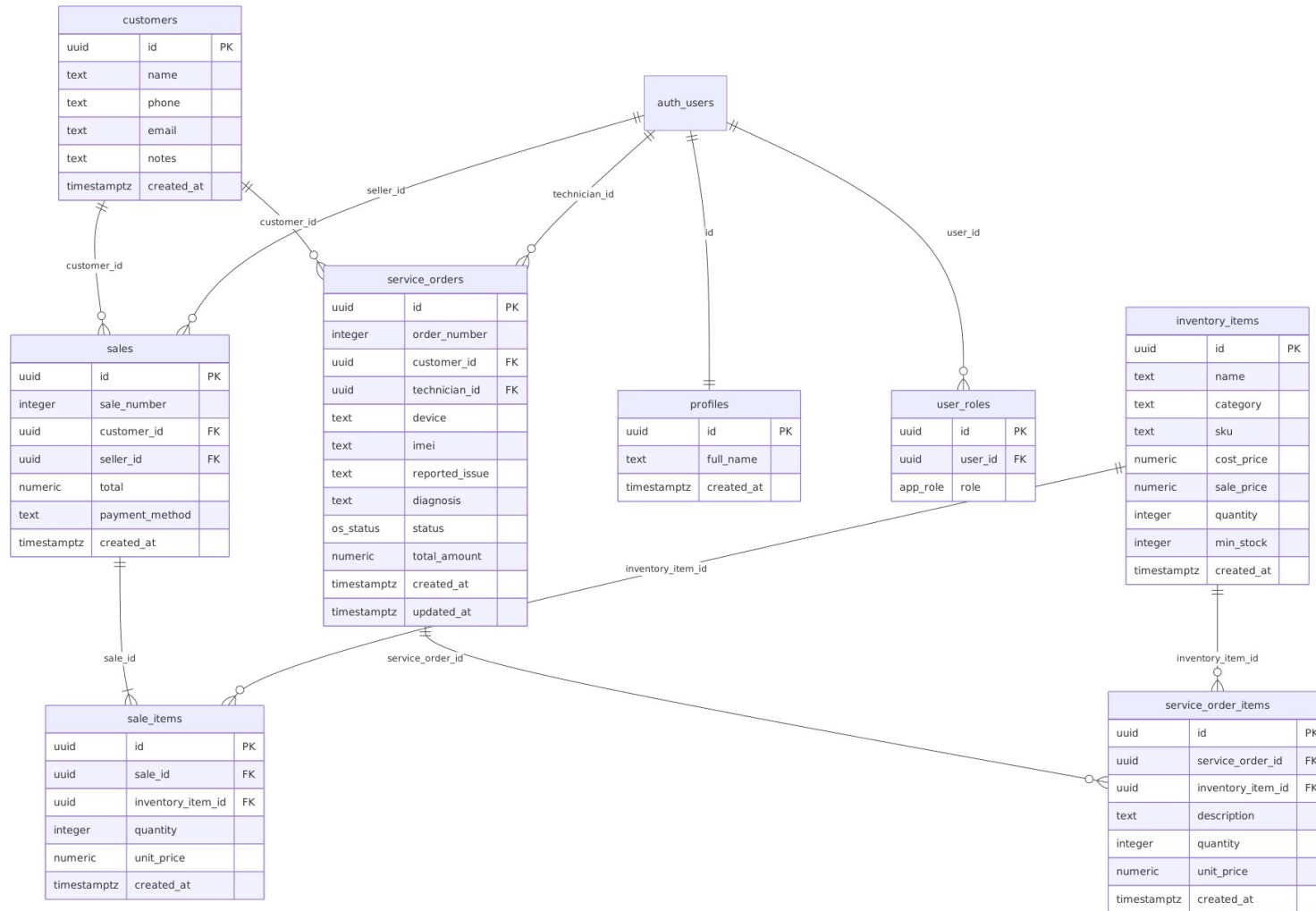


### **4.3 Modelo lógico do banco de dados**

O modelo lógico representa a estrutura mais próxima da implementação no banco de dados relacional. Nesse nível, são definidos os nomes das tabelas, os tipos de dados, as chaves primárias, as chaves estrangeiras e os relacionamentos. O projeto utiliza identificadores do tipo UUID para a maioria das tabelas, o que favorece unicidade e escalabilidade. Campos do tipo text armazenam descrições, nomes e observações, enquanto numeric é utilizado para valores monetários e timestamptz para registros de data e hora.

A estrutura também prevê tipos enumerados para padronizar informações sensíveis ao fluxo de negócio, como os papéis dos usuários e os status das ordens de serviço. O tipo app\_role define perfis como admin e atendente, enquanto o tipo os\_status define estados como aberta, em\_andamento, aguardando\_peca, pronta, entregue e cancelada. Essa abordagem reduz erros de digitação e impede o cadastro de valores inconsistentes.

Figura 3 - Modelo lógico do banco de dados do sistema TrinixTech



Fonte: Elaborado pelos autores (2026).

## 4.4 Descrição das principais tabelas

A seguir são descritas as principais tabelas que compõem a modelagem do banco de dados do sistema TrinixTech.

| Tabela              | Finalidade                                                                                                                                                |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| customers           | Armazena dados dos clientes, como nome, e-mail, telefone, observações e data de cadastro.                                                                 |
| inventory_items     | Registra produtos, peças e itens de estoque, incluindo categoria, SKU, preço de custo, preço de venda, quantidade disponível e estoque mínimo.            |
| service_orders      | Armazena as ordens de serviço abertas para manutenção, vinculando cliente, técnico, aparelho, IMEI, problema relatado, diagnóstico, status e valor total. |
| service_order_items | Relaciona uma ordem de serviço aos itens utilizados durante o atendimento, registrando descrição, quantidade e valor unitário.                            |
| sales               | Registra as vendas realizadas, vinculando cliente, vendedor, número da venda, valor total, forma de pagamento e data de criação.                          |
| sale_items          | Armazena os itens pertencentes a uma venda, relacionando produto, quantidade e preço unitário.                                                            |
| profiles            | Mantém dados complementares dos usuários autenticados, como nome completo e data de criação.                                                              |
| user_roles          | Define os papéis de acesso dos usuários do sistema, como administrador ou atendente.                                                                      |

## 4.5 Dicionário de dados

O dicionário de dados apresenta os principais campos identificados no modelo lógico, auxiliando na compreensão da estrutura e na padronização da documentação técnica.

### 4.5.1 Tabela customers

| Campo      | Tipo       | Descrição                           |
|------------|------------|-------------------------------------|
| id         | uuid       | Chave primária do cliente.          |
| name       | text       | Nome do cliente.                    |
| email      | text       | E-mail para contato.                |
| phone      | text       | Telefone do cliente.                |
| notes      | text       | Observações adicionais.             |
| created_at | timestampz | Data e hora de criação do registro. |

### 4.5.2 Tabela inventory\_items

| Campo      | Tipo    | Descrição                                   |
|------------|---------|---------------------------------------------|
| id         | uuid    | Chave primária do item de estoque.          |
| name       | text    | Nome do produto ou peça.                    |
| category   | text    | Categoria do item.                          |
| sku        | text    | Código de identificação do item no estoque. |
| cost_price | numeric | Preço de custo.                             |
| sale_price | numeric | Preço de venda.                             |
| quantity   | integer | Quantidade disponível em estoque.           |
| min_stock  | integer | Quantidade mínima para alerta de            |

|            |            |                                     |
|------------|------------|-------------------------------------|
|            |            | reposição.                          |
| created_at | timestampz | Data e hora de criação do registro. |

#### 4.5.3 Tabela service\_orders

| Campo          | Tipo           | Descrição                                    |
|----------------|----------------|----------------------------------------------|
| id             | uuid           | Chave primária da ordem de serviço.          |
| order_number   | integer/serial | Número sequencial da ordem de serviço.       |
| customer_id    | uuid           | Chave estrangeira para customers.            |
| technician_id  | uuid           | Identificador do técnico responsável.        |
| device         | text           | Aparelho recebido para manutenção.           |
| imei           | text           | Identificador do aparelho, quando aplicável. |
| reported_issue | text           | Problema relatado pelo cliente.              |
| diagnosis      | text           | Diagnóstico técnico.                         |
| status         | os_status      | Status atual da ordem de serviço.            |
| total_amount   | numeric        | Valor total da ordem de serviço.             |
| created_at     | timestampz     | Data de abertura da ordem.                   |
| updated_at     | timestampz     | Data da última atualização.                  |

#### 4.5.4 Tabelas sales e sale\_items

| Tabela     | Campo             | Tipo           | Descrição                        |
|------------|-------------------|----------------|----------------------------------|
| sales      | id                | uuid           | Chave primária da venda.         |
| sales      | sale_number       | integer/serial | Número sequencial da venda.      |
| sales      | customer_id       | uuid           | Cliente associado à venda.       |
| sales      | seller_id         | uuid           | Usuário responsável pela venda.  |
| sales      | total             | numeric        | Valor total da venda.            |
| sales      | payment_method    | text           | Forma de pagamento utilizada.    |
| sale_items | id                | uuid           | Chave primária do item da venda. |
| sale_items | sale_id           | uuid           | Venda relacionada.               |
| sale_items | inventory_item_id | uuid           | Produto vendido.                 |
| sale_items | quantity          | integer        | Quantidade vendida.              |
| sale_items | unit_price        | numeric        | Preço unitário do item vendido.  |

#### 4.5.5 Tabelas profiles e user\_roles

| Tabela     | Campo      | Tipo       | Descrição                                           |
|------------|------------|------------|-----------------------------------------------------|
| profiles   | id         | uuid       | Chave primária e referência ao usuário autenticado. |
| profiles   | full_name  | text       | Nome completo do usuário.                           |
| profiles   | created_at | timestampz | Data de criação do perfil.                          |
| user_roles | id         | uuid       | Chave primária do registro de papel.                |
| user_roles | user_id    | uuid       | Usuário vinculado ao papel.                         |
| user_roles | role       | app_role   | Perfil de acesso do usuário.                        |

#### 4.6 Relacionamentos entre as entidades



Os relacionamentos definidos no banco de dados permitem que os registros sejam vinculados de maneira lógica e consistente. A tabela `customers` relaciona-se com `sales` e `service_orders`, permitindo manter o histórico de compras e atendimentos de cada cliente. A tabela `sales` relaciona-se com `sale_items`, enquanto `sale_items` referencia `inventory_items`, possibilitando identificar quais produtos foram vendidos em cada transação.

De forma semelhante, `service_orders` relaciona-se com `service_order_items`, permitindo registrar as peças ou produtos utilizados em uma manutenção. A tabela `service_order_items` também referencia `inventory_items`, o que possibilita relacionar o consumo de estoque aos atendimentos técnicos realizados. Por fim, `profiles` e `user_roles` estão associados aos usuários autenticados, permitindo separar dados de identificação e controle de permissão.

| Relacionamento                         | Cardinalidade | Descrição                                                          |
|----------------------------------------|---------------|--------------------------------------------------------------------|
| customers -> sales                     | 1:N           | Um cliente pode possuir várias vendas registradas.                 |
| customers -> service_orders            | 1:N           | Um cliente pode possuir várias ordens de serviço.                  |
| sales -> sale_items                    | 1:N           | Uma venda pode conter vários itens.                                |
| inventory_items -> sale_items          | 1:N           | Um item de estoque pode aparecer em diferentes vendas.             |
| service_orders -> service_order_items  | 1:N           | Uma ordem de serviço pode conter vários itens utilizados.          |
| inventory_items -> service_order_items | 1:N           | Um item de estoque pode ser usado em diferentes ordens de serviço. |
| auth.users -> profiles                 | 1:1           | Um usuário autenticado possui um perfil complementar.              |
| auth.users -> user_roles               | 1:N           | Um usuário pode possuir registros de papel/permissão.              |

#### 4.7 Regras de integridade

As regras de integridade têm a finalidade de proteger a consistência dos dados armazenados. As chaves primárias garantem a identificação única de cada registro, enquanto as chaves estrangeiras asseguram que vínculos entre tabelas só sejam realizados com registros existentes. Por exemplo, uma venda deve estar vinculada a um cliente válido e seus itens devem referenciar produtos cadastrados em estoque.

Também devem ser aplicadas restrições para impedir valores inconsistentes. Campos monetários, como `total`, `sale_price`, `cost_price` e `unit_price`, devem aceitar valores numéricos adequados. Campos de quantidade, como `quantity` e `min_stock`, não devem permitir valores negativos. O status da ordem de serviço deve seguir os valores definidos no tipo `os_status`, evitando registros fora do fluxo operacional previsto.

- Toda venda deve possuir ao menos um item associado.
- Toda ordem de serviço deve estar vinculada a um cliente cadastrado.

- A baixa de estoque deve ocorrer quando houver venda ou utilização de item em ordem de serviço.
- O estoque mínimo deve apoiar alertas de reposição de peças ou produtos.
- Usuários devem ter permissões compatíveis com suas funções no sistema.

#### **4.8 Segurança, autenticação e controle de acesso**

A modelagem prevê separação entre autenticação e dados complementares do usuário. A autenticação pode ser gerenciada pelo serviço `auth.users`, enquanto a tabela `profiles` armazena informações de perfil e `user_roles` define os papéis de acesso. Essa separação é importante porque evita o armazenamento direto de senhas em tabelas próprias da aplicação e facilita o controle de permissões.

O controle de acesso deve limitar funcionalidades conforme o papel do usuário. Usuários com perfil de atendente podem registrar vendas, abrir ordens de serviço e cadastrar clientes. Usuários com perfil administrativo podem gerenciar estoque, cadastrar produtos e gerar relatórios. Essa estrutura contribui para reduzir riscos de alteração indevida de dados e melhora a rastreabilidade das operações realizadas no sistema.

Além disso, por armazenar dados pessoais de clientes, como nome, telefone e e-mail, o sistema deve adotar medidas de proteção compatíveis com a Lei Geral de Proteção de Dados. Entre as medidas recomendadas estão autenticação segura, restrição de acesso por perfil, uso de conexão criptografada, validação de entradas e armazenamento apenas dos dados necessários para a finalidade do sistema.

#### **4.9 Considerações sobre PostgreSQL e Supabase**

A estrutura apresentada é compatível com banco de dados PostgreSQL, especialmente pela utilização de tipos como `uuid`, `numeric`, `text` e `timestamptz`. Também há indicação de integração com Supabase, principalmente pela referência ao schema de autenticação `auth.users` e pelo uso de UUIDs. O PostgreSQL oferece recursos adequados para esse tipo de aplicação, como integridade referencial, tipos enumerados, funções de geração de identificadores e suporte a políticas de segurança em nível de linha.

No contexto do projeto TrinixTech, a adoção de uma base relacional contribui para a organização das informações e para a confiabilidade dos processos de atendimento técnico, vendas e controle de estoque. A modelagem também permite evolução futura do sistema, como inclusão de relatórios financeiros, histórico detalhado de manutenção, controle de garantia, movimentação de estoque e integração com comprovantes ou notas de atendimento.

## 4.10 Síntese da modelagem

A modelagem do banco de dados do TrinixTech atende às principais necessidades de uma assistência técnica, contemplando controle de clientes, vendas, estoque, ordens de serviço, usuários e permissões. A combinação entre DER, modelo lógico e casos de uso permite demonstrar a coerência entre os requisitos do sistema e a estrutura de persistência adotada. Dessa forma, o banco de dados contribui para a organização dos processos internos, melhora a confiabilidade das informações e permite a evolução futura da aplicação.

## APÊNDICE A SCRIPT SQL INICIAL

O script a seguir representa uma proposta inicial de implementação da estrutura apresentada no modelo lógico. Ele pode ser ajustado conforme as configurações específicas do ambiente de desenvolvimento e do banco utilizado no projeto.

```
-- Tipos enumerados
CREATE TYPE app_role AS ENUM ('admin', 'atendente');
CREATE TYPE os_status AS ENUM ('aberta', 'em_andamento', 'aguardando_peca',
'pronta', 'entregue', 'cancelada');

-- Clientes
CREATE TABLE customers (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name text NOT NULL,
    email text,
    phone text,
    notes text,
    created_at timestamptz DEFAULT now()
);

-- Itens de estoque
CREATE TABLE inventory_items (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    name text NOT NULL,
    category text,
    sku text UNIQUE,
    cost_price numeric(10,2) DEFAULT 0,
    sale_price numeric(10,2) DEFAULT 0,
    quantity integer DEFAULT 0,
    min_stock integer DEFAULT 0,
    created_at timestamptz DEFAULT now(),
    CONSTRAINT chk_inventory_quantity CHECK (quantity >= 0),
    CONSTRAINT chk_inventory_min_stock CHECK (min_stock >= 0)
);

-- Perfis de usuários autenticados
CREATE TABLE profiles (
    id uuid PRIMARY KEY REFERENCES auth.users(id),
    full_name text NOT NULL,
    created_at timestamptz DEFAULT now()
);

-- Papéis de usuário
CREATE TABLE user_roles (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES auth.users(id),
    role app_role NOT NULL
```

```

);

-- Ordens de serviço
CREATE TABLE service_orders (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    order_number serial UNIQUE,
    customer_id uuid NOT NULL REFERENCES customers(id),
    technician_id uuid REFERENCES auth.users(id),
    device text NOT NULL,
    imei text,
    reported_issue text NOT NULL,
    diagnosis text,
    status os_status NOT NULL DEFAULT 'aberta',
    total_amount numeric(10,2) DEFAULT 0,
    created_at timestamptz DEFAULT now(),
    updated_at timestamptz DEFAULT now(),
    CONSTRAINT chk_service_total CHECK (total_amount >= 0)
);

-- Itens utilizados em ordens de serviço
CREATE TABLE service_order_items (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    service_order_id uuid NOT NULL REFERENCES service_orders(id),
    inventory_item_id uuid REFERENCES inventory_items(id),
    description text,
    quantity integer NOT NULL DEFAULT 1,
    unit_price numeric(10,2) DEFAULT 0,
    created_at timestamptz DEFAULT now(),
    CONSTRAINT chk_service_item_quantity CHECK (quantity > 0),
    CONSTRAINT chk_service_item_price CHECK (unit_price >= 0)
);

-- Vendas
CREATE TABLE sales (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    sale_number serial UNIQUE,
    customer_id uuid REFERENCES customers(id),
    seller_id uuid REFERENCES auth.users(id),
    total numeric(10,2) DEFAULT 0,
    payment_method text,
    created_at timestamptz DEFAULT now(),
    CONSTRAINT chk_sales_total CHECK (total >= 0)
);

-- Itens de venda
CREATE TABLE sale_items (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    sale_id uuid NOT NULL REFERENCES sales(id),
    inventory_item_id uuid NOT NULL REFERENCES inventory_items(id),
    quantity integer NOT NULL DEFAULT 1,
    unit_price numeric(10,2) NOT NULL DEFAULT 0,
    created_at timestamptz DEFAULT now(),
    CONSTRAINT chk_sale_item_quantity CHECK (quantity > 0),
    CONSTRAINT chk_sale_item_price CHECK (unit_price >= 0)
);

```